Cryptography

6 – Digital Signatures

Gabriel Chênevert

October 3, 2025

Today

Elliptic curves

Digital signatures

Post-quantum cryptography

Generalized DLP

Let $(\mathcal{G}, \cdot)$ be a finite abelian group.

Given $g \in \mathcal{G}$ and x such that

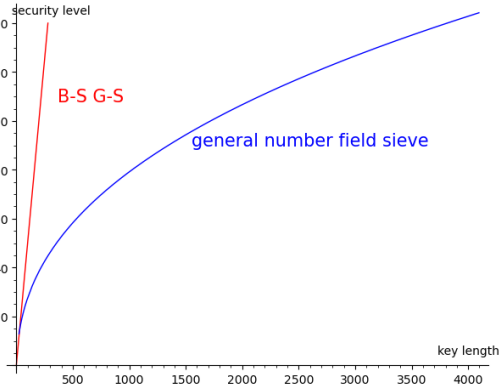
$$x = g^{\ell} = \underbrace{g \cdot g \cdots g}_{\ell} \quad \text{in} \quad \mathcal{G},$$

find $\ell \underset{n}{\equiv} \mathrm{dlog}_{\mathcal{G}}(x, g)$ where $n = \mathrm{ord}_{\mathcal{G}}(g)$, the smallest integer > 0 for which $g^n = 1$.

(Lagrange's theorem: we know in general that $\mathrm{ord}_{\mathcal{G}}(g)$ is a divisor of $|\mathcal{G}|$)

Best known DL algorithm: $\mathcal{O}(\mathrm{ord}_{\mathcal{G}}(g)^{\frac{1}{2}})$ for a generic group $\mathcal{G}$.

Recall


security level
140
120
100
80
60
40
20
B-S G-S
general number field sieve
key length
500
1000
1500
2000
2500
3000
3500
4000

Elliptic curves

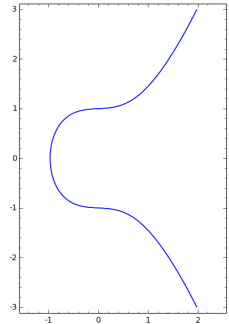
Definition

An *elliptic curve* in Weierstrass form is a plane curve defined by an equation of the form

$$\mathcal{E}: \ y^2 = x^3 + ax + b.$$

Example

$a = \frac{1}{10}$, $b = 1$



Some famous elliptic curves

Secp256k1

$$y^2 = x^3 + 7$$

Curve25519

$$y^2 = x^3 + 486662x^2 + x$$

Addition on an elliptic curve

Given $P, Q \in \mathcal{E}$, the line through P and Q intersects $\mathcal{E}$ at a third point, say $R = (x, y)$.

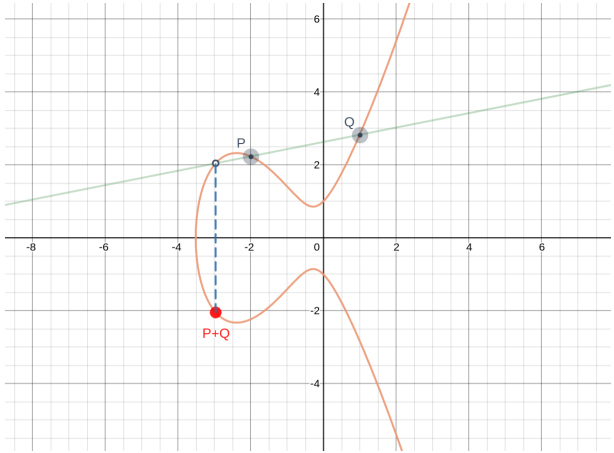
Definition

$$P + Q := (x, -y)$$

Fun fact: This makes $\mathcal{E}$ into an abelian group!

(The *point at infinity* $O = (?, \infty)$ being the neutral element)

Addition on an elliptic curve



DLP on an elliptic curve

Given $G \in \mathcal{E}$ of (additive) order n and $P \in \mathcal{E}$ such that

$$P = \ell G = \underbrace{G + \cdots + G}_{\ell} \quad \text{in} \quad \mathcal{E},$$

find $\ell \underset{n}{\equiv} \mathrm{dlog}_{\mathcal{E}}(P, G)$.

(Easy to solve over the real or complex numbers)

Elliptic curves over finite fields

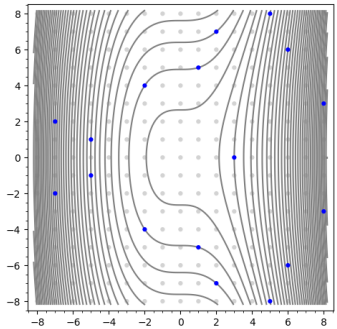
Instead: consider solutions modulo a fixed prime p

$$y^2 \underset{p}{\equiv} x^3 + ax + b$$

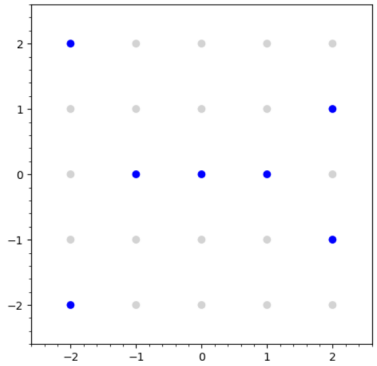
$\rightsquigarrow \mathcal{E}(\mathbb{F}_p)$ elliptic curve over the field with p elements

(a finite abelian group!)

$$y^2 \underset{17}{\equiv} x^3 - x$$

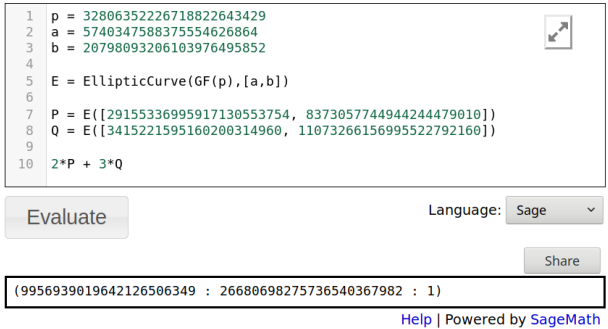


Example : $y^2 \underset{5}{\equiv} x^3 - x$



$$(-1,0) + (2,1) \underset{\mathcal{E}}{=} (-2,2)$$

Basic computations are easy...



...but the DLP is hard!

Size of $\mathcal{E}$

Theorem (Hasse bound)

$$\#\mathcal{E}(\mathbb{F}_p) = 1 + p + \mathcal{O}(\sqrt{p})$$

hence $\#\mathcal{E}(\mathbb{F}_p) \approx p$.

We use elliptic curves with points G of large order $n \approx p$.

Example: ECDH

- Alice and Bob agree on "safe" parameters $\mathcal{E}$ and G.
- Alice chooses a, computes $A = aG$ in $\mathcal{E}$.
- Bob choooses b, computes $B = bG$ in $\mathcal{E}$.
- Shared secret is

$$K := (ab)G = aB = bA.$$

Parameter generation

To get ℓ bits of security:

- choose a 2ℓ-bit prime p
- an elliptic curve $\mathcal{E}$ over $\mathbb{F}_p$
- and a point G on $\mathcal{E}$ of (almost) prime order n that generates (most of) $\mathcal{E}(\mathbb{F}_p)$.

Much harder to manufacture than *e.g.* for RSA – but can be reused.

Recommended curves

In the US, NIST proposed in 2005 a list of 5 elliptic curves of size

$$192,\ 224,\ 256,\ 384 \text{ and } 521 \text{ bits}$$

(**P-256** and up are still recommended)

Alternatives:

- Brainpool curves
- **Curve25519**, a curve with efficient operations offering 128 bits of security
- **Curve448**, a similar curve with higher security level

Today

Elliptic curves

Digital signatures

Post-quantum cryptography

Recall: message authentication

To authenticate a message m with a shared secret key k,

- Alice appends to it a tag $t = \mathrm{MAC}(k, m)$;
- upon reception of (m, t), Bob checks whether

$$t \stackrel{?}{=} \mathrm{MAC}(k, m).$$

Provides security against *forgery* by a malicious third party.

The problem with MACs

Alice and Bob share the exact same capabilities, so this system cannot protect them *against one another*.

Forgery:

Bob: "My name is Alice and I will give 100€ to Bob." ✗

Repudiation:

Alice: "My name is Alice and I will give 100€ to Bob." ✓

Alice: "Hey I never said that! It was Bob!" ✗

Digital signature provides

- sender authentication
- message integrity
- binding between message and sender
- non-falsification/forgery
- non-repudiation

Applications

- approval/agreement (contracts)
- authenticity of official documents
- software distribution
- financial transactions
- IoT
- ...

The (cryptographic) notion of *digital signature* should not be confused with the closely related (legal) notion of *electronic signature* (*cf.* European eIDAS regulation)

Construction idea

Use public-key encryption "in reverse"!

- private encryption (signing) key $k_{\text{priv}} = k_e$
- public decryption (verification) key $k_{\text{pub}} = k_d$

only Alice can sign, anyone can verify

Protocol (1^{st} try)

To sign a message m with private key k_e:

- Alice appends to it $s = E(k_e, m)$.

Upon reception of a pair (m, s):

- Bob checks with associated public key whether

$$D(k_d, s) \stackrel{?}{=} m.$$

Problems

- Asymmetric ciphers are inherently slow:

  problematic for long messages
- Need to use "multiple blocks" version of encryption
- Signed message is twice as long as original message!

Solution: sign a hash

To sign a message m with private key k_e:

- Alice computes $h = H(m)$;
- appends $s = E(k_e, h)$ to m.

Upon reception of a pair (m, s):

- Bob checks with associated public key whether

$$D(k_d, s) \stackrel{?}{=} H(m).$$

To sum up:

Digital signatures are (usually) built from a hash function + asymmetric encryption.

(**hash-then-sign paradigm**)

- Only Alice can sign with private $k_{\text{priv}} = k_e$.
- Anyone can check that the signature is genuine using public $k_{\text{pub}} = k_d$.
- Footprint is minimal (computation time + size of signed message).

Note: any weakness in the hash *or* encryption directly impacts the security of the signature.

Definition

A **digital signature scheme** consists of a pair of algorithms:

- **signature** $S(k_{\text{priv}}, m)$
- **verification** $V(k_{\text{pub}}, m, s) \in \{0, 1\}$

(along with a **key generation** algorithm that produces valid pairs $(k_{\text{priv}}, k_{\text{pub}})$)

In practice

Most digital signature schemes in use today are based on RSA or DLP ciphers.

Warning 1

Signatures do nothing to conceal the content of the message; encryption needs to be used for that as well.

Warning 2

Never use the same key pairs for encryption and signature! Use a dedicated KDF to derive both from a master key if needed.

RSA Probabilistic Signature Standard

Specified in PKCS #1

- H is taken to be one of the flavors of SHA-2
- The actual value that is signed incorporates some random salt
- Signature (encryption) can be sped up using CRT
- Verification (decryption) can be sped up by using a small Fermat prime

RSA-PSS protocol (1/2)

To sign a message m with public n, d, private e, Alice:

- computes $h = \mathrm{SHA2}(m)$
- chooses random salt k
- applies padding $M = T(k, h) \in [\![0, n[\![$
- appends $s :\underset{n}{\equiv} M^e$

RSA-PSS protocol (2/2)

Upon reception of a pair (m, s), Bob:

- computes $h = \mathrm{SHA2}(m)$
- decrypts $M \underset{n}{\equiv} s^d$
- recovers random salt k from M
- checks whether

$$M \overset{?}{=} T(k, h)$$

Digital Signature Standard

NIST specifies three other signature schemes in the Digital Signature Standard:

- **DSA** (variant of mod n ElGamal)
- **ECDSA** (DSA over elliptic curves)
- **EdDSA** (better DLP-based signature scheme using elliptic curves)

(Probabilistic padding not needed).

DSA (1/2)

Public parameters: (can be reused)

- a medium-sized prime $q \approx 2^{256}$
- a large prime $p \approx 2^{2048}$ such that $q \mid p - 1$
- an integer g of multiplicative order q mod p:

$$g^q \underset{p}{\equiv} 1$$

Keys:

- private $x \in]\!]0, q[\![$
- public $y \underset{p}{\equiv} g^x$

DSA (2/2)

Signature:

- Choose random $k \in]\!]0, q[\!]$
- Compute $r = (g^k \,\%\, p) \,\%\, q$
- Compute $s \underset{q}{\equiv} k^{-1} \cdot (H(m) + xr)$. Signature is (r, s).

Verification: upon reception of (m, r, s),

- Compute $t \underset{q}{\equiv} s^{-1}$
- Verify if

$$\left((g^{H(m)} y^r)^t \,\%\, p\right) \,\%\, q \overset{?}{=} r.$$

Using elliptic curves

(EC)DSA signatures are more compact than RSA

but: no formal security proof exists!

The crypto community today favors using some form of *Schnorr signature*

e.g. Edwards-curve Digital Signature Algorithm (Ed25519)

with actual formal reduction to hardness of DLP (Seurin 2012).

Schnorr signature algorithm (1/2)

Parameters:

- a group $\mathcal{G}$ of prime order q for which the DLP is hard
- a generator g of $\mathcal{G}$
- a secure hash function $H : \{0,1\}^* \to [\![0, q[\![$

Keys:

- private $x \in [\![0, q[\![$
- public $y = g^x$.

Schnorr signature algorithm (2/2)

Signature of a message m:

- choose random $k \in]\!]0, q[\![$
- compute $e = H(g^k \| m)$, $s = k - xe$
- signature is (s, e)

Verification:

- Check if $H(g^s y^e \| m) \stackrel{?}{=} e$

The problem with public keys

Bob can check Alice's signature provided he has her public key.

Alice can broadcast it publicly...

...but how to prevent man-in-the-middle attacks?

Back to square one! (again)

Certificate

A trusted third party (Trent) *certifies* the pair (Alice, k_{pub}) by broadcasting:

"I, Trent, certify that Alice's public key is k_{pub}."

Revised protocol

To sign a message m, Alice:

- computes $s = S(k_{\text{priv}}, m)$
- sends (m, s) along with her certificate for k_{pub}

Bob:

- checks that the certificate is valid
- verifies the signature using k_{pub}

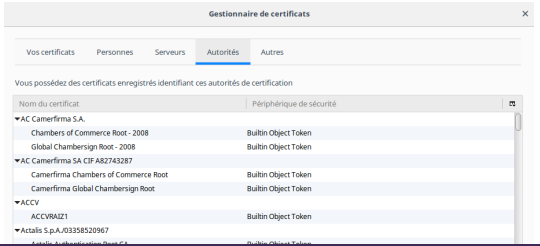
Trust management

Two main approaches:

- web of trust (*e.g.* PGP)
- public-key infrastructure (*e.g.* X.509):

  chain of certification authorities (CAs), revocation lists ...

Trusted top level CAs: `/etc/ssl/certs` (Linux), `certmgr.msc` (Windows)



Today

Elliptic curves

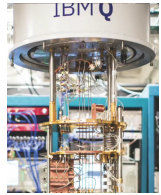
Digital signatures

Post-quantum cryptography

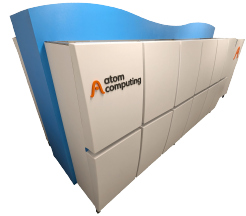
Quantum computers

State of the art as of 2025:

Simulated annealing (5000 qubits)



IBM: 1121 qubits



Atom: 1180 qubits

Large-scale quantum computers *might* become a reality in 25-30 years

Grover's algorithm

Grover's algorithm

aka Better brute force search

Given an arbitrary function $f : A \to B$ with $|A| = n$ and $b \in f(A)$, looking for a preimage $a \in A$ such that $f(a) = b$ classically takes n evaluations.

A quantum computer can (*with high probability*) find one with only $\sqrt{n}$ evaluations.

$\implies$ symmetric keys will need to be twice as long (ok: move to AES-256)

Shor's algorithm

Quickly finds periods or arbitrary functions.

Look for integers a for which $f(x) \underset{n}{\equiv} a^x$ has even period r.

$$(a^{\frac{r}{2}})^2 \underset{p \cdot q}{\equiv} 1 \implies \begin{cases} a^{\frac{r}{2}} \underset{p}{\equiv} \pm 1 \\ a^{\frac{r}{2}} \underset{q}{\equiv} \pm 1 \end{cases}$$

There is then a 50% chance that

$$\gcd(a^{\frac{r}{2}} - 1, n) \quad \text{and} \quad \gcd(a^{\frac{r}{2}} + 1, n)$$

are non-trivial factors of n.

Shor's algorithm

Factors n in time

$$(\log n)^2(\log\log n)(\log\log\log n) \in \mathrm{Poly}(\log n)$$

~~RSA~~ ~~(EC)DSA~~ ~~EdDSA~~ ~~(EC)DH~~

Post-quantum encryption

It is necessary to start thinking today about potential alternatives that could be both efficient *and* secure against a quantum opponent.

4 main leads:

- hash-based encryption
- based on error-correcting codes
- lattice-based
- hidden field equations.

NIST PQC

In 2024, NIST standardized two quantum-resistant signature algorithms:

- ML-DSA (previously known as CRYSTALS-Dilithium)
- SLH-DSA (previously known as SPHINCS$^+$)

as well and alternative lattice-based signature scheme (previously known as FALCON) and a key establishment primitive ML-KEM (previously known as CRYSTALS-Kyber)

2025 : HQC, another key establishment scheme, announced for standardization

Lattice-based encryption

Consider a lattice $\Lambda = \{ \sum_i a_i \mathbf{v}_i \mid a_i \in \mathbb{Z}\}$ where $\mathbf{v}_1, \ldots, \mathbf{v}_n$ are linearly independant vectors in $\mathbb{R}^n$.

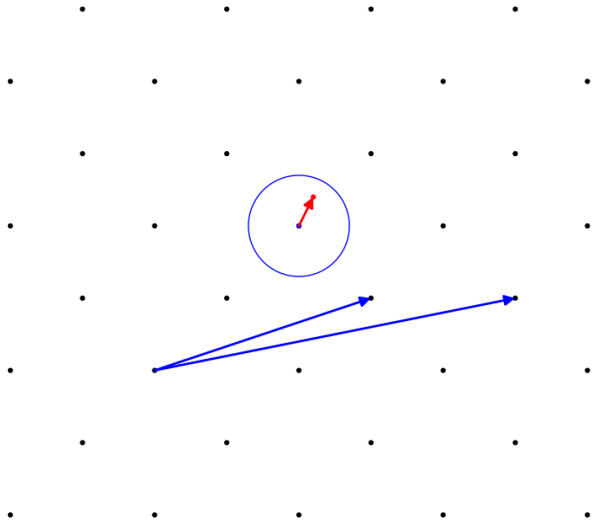
Encryption: we add to a message $\mathbf{v} \in \Lambda$ a small perturbation $\mathbf{e} \in \mathbb{R}^n$,

$$\mathbf{c} = \mathbf{v} + \mathbf{e}.$$

Decryption: given $\mathbf{c}$, look for the nearest vector $\mathbf{v}$ in the lattice.

Algorithmically difficult problem if an adapted basis is not known.

Lattice-based encryption



Learning with errors (LWE)

A similar problem :

given data points $(\mathbf{x}_i, y_i)$ with $\mathbf{x}_i \in \mathbb{R}^n$ and $y_i \in \mathbb{R}$, find a value of $\mathbf{a} \in \mathbb{R}^n$ for which

$$y_i = \mathbf{a} \cdot \mathbf{x}_i \qquad (i = 1, \ldots, n).$$

(a typical problem in machine learning)

Becomes significantly harder (even for a quantum computer) when noise is added :

$$y_i = \mathbf{a} \cdot \mathbf{x}_i + \mathbf{e}_i.$$